Instructions:

- Task naming format: fullID_T01L03_2A.c/CPP
- If you find any issues in problem description/test cases, comment in the google classroom.
- If you find any test case that is tricky that I didn't include but others might forget to handle, please comment! I'll be happy to add.
- Use appropriate comments in your code. This will help you to easily recall the solution in the future.
- Obtained marks will vary based on the efficiency of the solution.
- Do not use <bits/stdc++.h> library, add the necessary libraries one after another.
- Do not use any built-in functions or any STL libraries. If the solution is extremely difficult without the usage of such components, discuss it with the instructors.
- Modified sections will be marked with BLUE color.

Task-01:

Assume your program will take a set of positive integers(1~9) as input and store them in an array. It will stop taking input for any negative number or zero.
Now, the program will show how many times each of the numbers have occurred. (Don't show anything for a number that didn't occur.)

Input	**Output**
3 4 5 7 8 9 2 3 3 4 4 5 0	3 occurs 3 times 4 occurs 3 times 5 occurs 2 times 7 occurs 1 time 8 occurs 1 time 9 occurs 1 time 2 occurs 1 time
5 2 2 3 3 3 1 1 9 -1	3 occurs 3 times 2 occurs 2 times 1 occurs 2 times 5 occurs 1 time 9 occurs 1 time

[**Note**: Output is arranged based on the occurrence of each number. If the frequency of two numbers is the same, it's not mandatory to sort based on the relative ordering.]

Note: STL libraries are not allowed for this task. Implement the sort function manually.

Task 2:

IUT hired a group of workers to build a new dormitory. The energy level of each worker varies.
Each day, they have new tasks in hand where they have to nominate two workers to work in a pair. The task has a defined energy level (let's say, 'n'). So two workers with energy levels n_1, and n_2 have to be selected such that $n=n_1+n_2$.

Your task is to find the number of possible pairs of workers that can be nominated to solve the task.

Notes:

- Assuming the serial of worker starts from 1
- Stop taking input when you get 0
- The energy level of workers can be positive/negative (except zero).
- In the output, you have to show the number of pairs and the workers who form the pairs. (worker1,worker-2) means the same pair as (worker-2,worker-1) so don't consider both of them!

Input	**Output**
1 5 7 -1 0 6	2 Pairs: (worker-1, worker-2) (worker-3, worker-4)
1 5 7 -1 5 0 6	3 Pairs: (worker-1, worker-2) (worker-3, worker-4) (worker-1, worker-5)
1 1 1 1 0 2	6 (worker-1, worker-2) (worker-1, worker-3) (worker-1, worker-4) (worker-2, worker-3) (worker-2, worker-4) (worker-3, worker-4)
10 12 15 -1 7 6 5 4 2 1 1 1 0 11	6 pairs (worker-1, worker-10) (worker-1, worker-11) (worker-1, worker-12) (worker-2, worker-4) (worker-5, worker-8) (worker-6, worker-7)
1 5 3 7 -1 0	2 Pairs:

6	(worker-1, worker-2) (worker-4, worker-5)
1 5 7 -1 0 5	No pair found!

Task 03:
The Fibonacci sequence is a series where the next term is the sum of previous two terms. The first two terms of the Fibonacci series are 0 followed by 1.
The Fibonacci sequence: 0, 1, 1, 2, 3, 5, 8, 13, 21 …

Given a number ‘n’, your task is to write a function for calculating the n-th Fibonacci number of the series.
Solve the problem in both **iterative** and **recursive** approaches.

Input	**Output**
10	34
20	4181
50	7778742049
100	218922995834555169026

Task 04:

Xenia the beginner mathematician is a third-year student at elementary school. She is now learning the addition operation.

The teacher has written down the sum of multiple numbers. Pupils should calculate the sum. To make the calculation easier, the sum only contains digits. Still, that isn't enough for Xenia. She is only beginning to count, so she can calculate a sum only if the summands follow in non-decreasing order. For example, she can't calculate the sum 1+3+2+1 but she can calculate sums 1+1+2 and 3+3.

You've got the sum that was written on the board. Rearrange the terms and print in such a way that Xenia can calculate.

Input	**Output**
3+2+1	1+2+3
1+1+3+1+3	1+1+1+3+3
2	2
5+2+2+4	2+2+4+5